# Intro to Software English

## A CONTROLLED LANGUAGE FOR PROSE ABOUT SOFTWARE

Why agents write the way they write, what aerospace did about the same problem in 1986, and a plugin that checks every commit message.

# A paragraph an agent wrote

> Let's dive into the Messages resource — the beating heart of the Gmail API. Essentially, a message is an email, and this resource is how you list them, read them, and send them. Simple as that. Under the hood, Gmail wants every message to live inside a thread, so you'll notice each message carries a `threadId` alongside its own `id` — think of the thread as the envelope and the message as the letter inside.

# Every fact in the agent's paragraph is correct

**48**

errors found by a script, in a reference document of 108 lines

**0**

facts wrong

The 48 errors are in the words around the facts.

PART 1 OF 4

# Agents have habits

# Seven habits recur in agent prose

| Habit | Example from the Gmail reference |
| --- | --- |
| Filler | `Essentially`, `simply`, `genuinely`, `obviously` |
| The em dash | `the Messages resource — the beating heart` |
| Metaphor | `the thread as the envelope and the message as the letter` |
| A human trait for a system | `Gmail wants`, `the API knows who you are`, `Gmail is smart enough` |
| Rhythm | `Simple as that.` `No manual bookkeeping required.` |
| Contrast | `It's not about which format is "best" — it's about ...` |
| Narration | `Let's dive in`, `More on that in a sec`, `bear with me` |

# Each habit costs time, trust, or a fact

- **Time.** A reader parses the rhythm and the metaphor, then discards both to find the fact.
- **Trust.** A reader who recognises a habit reads the rest with less trust, before any fact is checked.
- **Facts.** A sentence written for rhythm can state nothing. The reader cannot tell until the end of it.

The cost repeats. Agents read prose that agents wrote, and the next document has the same habits.

PART 2 OF 4

# Aerospace controlled its prose in 1986

# ASD-STE100

**Simplified Technical English** is a standard of the European aerospace industry. It was first issued in 1986, for maintenance manuals read by people whose first language is not English.

Two parts:

- **Writing rules.** Short sentences, one instruction per sentence, active voice, a fixed sentence length limit.
- **A closed dictionary.** Each approved word has one meaning and one part of speech. A word outside the dictionary is not used.

The aim of the standard is a manual that reads the same from every writer.

# A closed vocabulary makes prose uniform, whoever writes it

The writers were human engineers and the readers were human mechanics. A word list and a checker enforced the rules.

The approach applies when the writer is a model.

PART 3 OF 4

# Software English

# A controlled natural language for software prose

Software English covers prose about software and systems:

- documentation, READMEs, ADRs, API references;
- commit messages and PR descriptions;
- code comments;
- an agent's own chat replies.

The primary reader of Software English prose is a person. The restricted grammar and the closed vocabulary also make the prose checkable by a script.

Software English takes its approach from the field of controlled languages. It is independent of ASD-STE100 and built for the software domain.

# Agents may also read Software English more reliably

An agent that reads the prose as context has fewer word senses to disambiguate and one fact in each sentence to track. That may reduce its reading errors.

The spec states this as a likely benefit, not a design goal. The software-english repository has no measurement of it yet.

# Two tiers

| Tier | Who checks it | Examples |
|---|---|---|
| **Deterministic** | A script. A word list or a pattern is enough. | Banned words, the em dash, continuous tense, a fixed anthropomorphism list |
| **Inference** | A model. The rule needs a judgement about meaning. | A paraphrased metaphor, a sentence written for rhythm, a contrastive frame |

A rule is in the deterministic tier only when a lookup or a regular grammar checks it with no judgement about meaning.

The rule catalogue has 36 rules today. 6 are deterministic.

# Deterministic: banned words, with the fix

| Word | Fix |
| --- | --- |
| `simply`, `essentially`, `really`, `obviously` | Cut it |
| `robust`, `seamless`, `powerful`, `elegant` | Cut it |
| `leverage` | use |
| `surface` (as a verb) | show, return, or report |
| `carries`, `holds` | has, contains, or includes |
| `reach`, `land`, `flow through` | name the action: read, write, send |

The list exists for words a writer uses out of habit, so the checker gives a direct fix, not only a report.

# Deterministic: the em dash, continuous tense, and human traits

**The em dash.** The rule has no exception. An em dash most often joins two clauses without stating how they relate.

> The prose is dense — read it twice.

**Continuous tense for a system.** A system `runs`, not `is running`.

> The worker is polling the queue.

**A human trait for a system.** A verb on a fixed list fails unless a person is the subject: `wants`, `knows`, `decides`, `tries`, `expects`.

> Gmail wants every message to live inside a thread.

# Inference: the rules a script cannot decide

**Metaphor, paraphrased.** No fixed word marks it, so a model has to recognise it.

> Think of the cache as a waiting room for data.

**A cadence device.** A sentence written for rhythm states nothing new. Delete it and no fact is lost.

> The consumer retries three times, then writes to the dead-letter topic.
> No silent drops.

**Contrastive framing.** The sentence denies a claim nobody made.

> It's not about which format is best. It's about what you do with it.

# Each habit has the same fix: state what the system does

| Instead of | Write |
| --- | --- |
| `Gmail wants every message to live inside a thread` | `Each message has a threadId` |
| `the API knows who you are from the access token` | `me names the user that the access token identifies` |
| `Gmail is smart enough to figure out the rest` | `The Subject header must match the thread` |
| `take it with a pinch of salt` | `The value is not exact` |

Each fix is shorter than the text it replaces, and each one states a fact that text only implied.

# Document types

A type governs structure, on top of the sentence rules. 15 types today: RFC, ADR, Design, API Design, PR Description, Specification, Technical Manual, the four Diátaxis types, and four more.

A **Reference** describes and only describes. It has no open question, no "next steps", and no pointer to a plan.

> **What's next**
> We'll add a section on the `modify` and `trash` endpoints in a follow-up, and there's an open question about whether batch requests deserve their own page. Watch this space.

That section is a fault in a Reference, whatever its wording.

# Before and after: the opening of the Gmail reference

## BEFORE

Let's dive into the Messages resource — the beating heart of the Gmail API. Essentially, a message is an email, and this resource is how you list them, read them, and send them. Simple as that. Under the hood, Gmail wants every message to live inside a thread, so you'll notice each message carries a `threadId` alongside its own `id`.

## AFTER

A message is one email. Each message has an `id` and a `threadId`. The `threadId` names the thread that contains the message. The resource has three operations: list, get, and send.

# Before and after: what the list endpoint returns

## BEFORE

Here's the thing: the list endpoint doesn't actually return messages. It returns message *references* — just an `id` and a `threadId` for each one. No subjects, no bodies, nothing. This trips up a lot of people, so it's worth noting up front.

## AFTER

`GET /gmail/v1/users/{userId}/messages` returns references to messages. A reference has two fields, `id` and `threadId`, and no other field. A separate get request returns the content of a message.

# The Software English version of the Gmail reference has 26 fewer lines and no errors

### BEFORE

- 108 lines
- 48 deterministic errors
- A "What's next" section in a Reference

### AFTER

- 82 lines
- 0 errors, both tiers
- Every fact from the before version

Both documents are in the software-english repository, beside this deck.

PART 4 OF 4

# The plugin

# swe, a Claude Code plugin

```
/plugin marketplace add jimbarritt/claude-plugins
/plugin install swe@jimbarritt-claude-plugins
```

No further setup is needed. The plugin also runs under GitHub Copilot CLI.

The plugin fetches its rule data from the Software English repository, so the plugin and the spec change together.

# Three hooks check commits, artifacts, and messages

Each hook runs before an action and blocks it on a deterministic error:

| Hook | What it reads |
| --- | --- |
| `bash-check` | A `git commit` message, a `gh pr` or `gh issue` body |
| `artifact-check` | A file about to publish as an artifact |
| `mcp-send-check` | An outbound message to Slack, Gmail, or Google `Drive` |

A blocking hook prints the rule, the flagged text, and the fix. The agent reads the report, edits the text, and runs the action again.

# Three checks you select yourself

- **The output style.** `/output-style`, then "Software English".
  Claude Code sends the rules with every request, so the rules apply while the prose is written.
  Cost: 6% to 11% more input tokens per turn.
- **`/swe:lint-file <path>`.** Both tiers, on one file, now.
- **`/swe:install-commit-hook`.** A `pre-commit` hook that blocks a commit of a Markdown file with no clean lint record for that exact content.

# How the inference tier runs

There is no second model and no API call.

1. The script lists the file as a numbered worklist of prose units: one per heading, paragraph, list item, and table.
2. It prints the inference rules that apply, with located candidates where a pattern gives a hint.
3. Claude judges each unit against each rule, as part of its own turn.
4. Claude's verdict is recorded against the file's exact content. One edit invalidates it.

The commit hook reads that record. A file edited after its last lint pass blocks the commit.

# Feedback from real use improves the spec

```
/swe:feedback false-positive   "flagged 'open' as a verb in 'the open connection'"
/swe:send-feedback             files a GitHub issue for each real pattern
```

An unattended session takes an issue, writes a rule, adds a test, and releases both the spec and the plugin.

Since September 2026, more than thirty issues went through this process across the two repositories. Nineteen of them were on the spec itself, and most of those added or widened a rule.

# Install swe and lint one file this week

1. Install the plugin.
2. Run `/swe:lint-file` on one README you wrote by hand.
3. Select the output style and ask for one commit message.

Then read the report. Each line names the rule, the flagged text, and the fix.

# Links

- Spec: **github.com/jimbarritt/software-english**
- Plugin: **github.com/jimbarritt/claude-plugins**
- This deck and both samples: `docs/public/presentations/intro` in the software-english repository

COLOPHON

# Claude wrote the first draft, and it broke the rules the deck describes

- **A withholding heading.** A reviewer found `The habits, named`. A full review then found ten more headings with a similar fault.
- **A missing point.** The deck omitted that agents may also read the prose more reliably.
- **A metaphor.** A habit was named `A mind in the machine`.
- **Seven errors.** The first deterministic run flagged seven.
- **Two figures with no source.** Both were removed.